



System Development Life Cycle (SDLC)

The **System Development Life Cycle (SDLC)** is a structured framework of seven phases—**Planning, Analysis, Design, Development, Testing, Implementation, and Maintenance**—that ensures software is delivered efficiently, meets user needs, and remains reliable over time.

It provides a road map for teams to manage **complexity**, **reduce risks**, and **align projects** with business goals.



Planning

This phase defines the project's scope, objectives, resources, and timeline. Feasibility studies (technical, financial, operational) are conducted to ensure the project is viable.

- Define project scope, goals, timeline, and resources.
- Conduct feasibility studies (technical, financial, operational).

Output: Project plan and feasibility report.



Analysis

Detailed requirements are gathered from stakeholders and documented. The goal is to understand **what** the system must achieve, resulting in a Software Requirement Specification (SRS).

- Gather and validate requirements from stakeholders.
- Document functional and non-functional needs.

Output: Software Requirement Specification (SRS).



Design

Requirements are translated into system architecture and design blueprints. High-level design covers modules and databases, while low-level design specifies workflows, APIs, and logic.

- Convert requirements into system architecture and blueprints.

High-Level Design (HLD) : architecture, modules, database.

Low-Level Design (LLD) : workflows, APIs, logic.

Output : Design Document Specification (DDS).



Development

The actual coding of the system takes place here. Developers build modules according to design documents, perform unit testing, and integrate components.

- Code the system based on design documents.
- Perform unit testing and code reviews.

Output: Source code and executable application.



Testing

The system is tested for functionality, performance, and security. This includes unit, integration, system, and user acceptance testing (UAT) to ensure quality.

- Verify system functionality, performance, and security.

Types : Unit, Integration, System, and User Acceptance Testing (UAT).

Output : Test cases, defect reports, quality metrics.



Implementation

The system is deployed into production. Activities include installation, configuration, data migration, and user training. Modern approaches often use CI/CD pipelines for faster releases.

- Deploy the system into production.

Activities : Installation, configuration, data migration, user training.

Modern approach : CI/CD pipelines for faster releases.

Output : Live application.



Maintenance

After deployment, the system requires ongoing support. This includes bug fixes, updates, performance tuning, and adding new features to meet evolving needs.

- Provide ongoing support after deployment.

Activities : Bug fixes, updates, performance tuning, new features.

Output : Patches, upgrades, and improved versions.



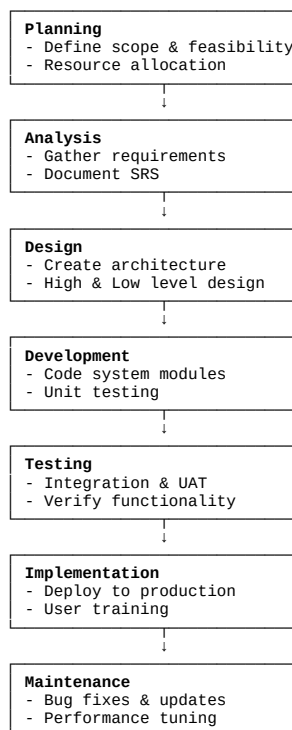
Quick Reference

— The SDLC ensures projects are **structured, predictable, and aligned with business goals**. Whether using **Waterfall (sequential)** or **Agile (iterative)** models, the phases remain the backbone of software development, helping teams deliver **high-quality, cost-effective, and user-centric solutions**.

<u>Phase</u>	<u>Purpose</u>	<u>Key Deliverable</u>
Planning	Define scope, feasibility	Project Plan
Analysis	Gather requirements	SRS
Design	Create system blueprint	DDS
Development	Build the system	Source Code
Testing	Validate quality	Test Reports
Implementation	Deploy system	Live Application
Maintenance	Support & improve	Updates/Patches

✨ — This flow shows the continuous lifecycle of software development:

Planning → Analysis → Design → Development → Testing → Implementation → Maintenance.



Each phase builds on the previous one, ensuring projects are structured, predictable, and aligned with business goals.